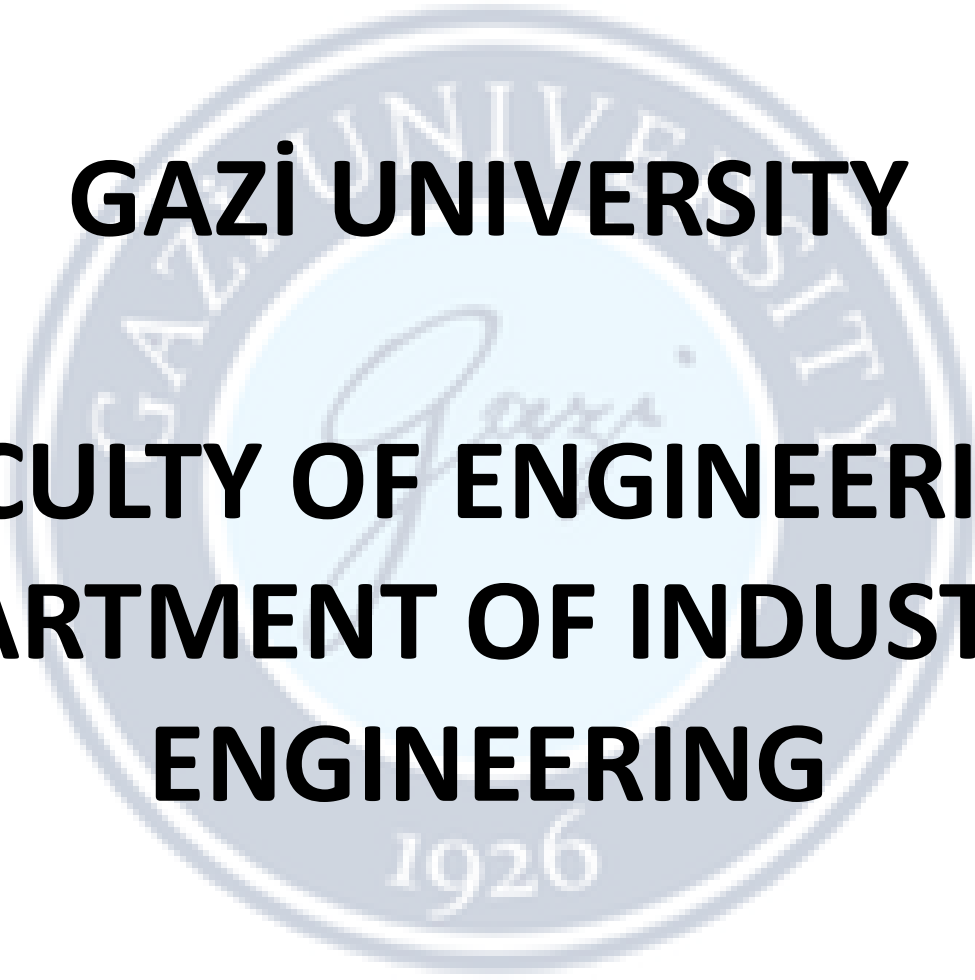


The logo of Gazi University is a circular seal. It features the university's name in Turkish and English around the perimeter, with the founding year '1926' at the bottom. In the center is a stylized signature of 'Gazi'.

GAZİ UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE326-DATABASE MANAGEMENT SYSTEMS

WEEK 7: SQL Studio and PostgreSQL CREATE COMMAND



PostgreSQL

The World's Most Advanced Open-Source Relational Database

Database Management Systems

SQL STUDIO

Announcements

LATEST ANNOUNCEMENT

11 Mar 2026

SQL Studio Live: The course web SQL tool is now live here: [SQL Studio](#)

Aiven Account and Connection String: If you need a free PostgreSQL instance and a valid connection string, follow this guide: [Aiven Guide](#)

Windows Local Installation: Official PostgreSQL Windows installer: [PostgreSQL for Windows](#) and official pgAdmin Windows installer: [pgAdmin 4 for Windows](#)

Recent Installation Video: [How to install PostgreSQL and pgAdmin 4 on Windows 11 \[2025\]](#)

pgAdmin Tutorial: [pgAdmin 4 Tutorial for Beginners](#)

LATEST ANNOUNCEMENT

01 Mar 2026

Final Project Published: DBMS Final Project details are now available here: [DBMS Final Project Details Page](#)

Submission Type: Submission-only project (no in-class presentation/demo). Visit following page to submit your group info: [Submission Form](#)

Group Size: Up to 4 students per project group.

Deadline: Friday, 22 May 2026, 23:59 (Türkiye Local Time).

09 Feb 2026

Lecturer: Dr. Ercan EZİN

Course Grade Distribution: %50 Finals, %30 Midterm, %20 Projects

Attendance: Attendance is mandatory for at least %70 of the weeks where there is a slide in the slides column

Contact: Please send an email to dr.ercan.ezin@gmail.com for any questions or concerns with your name and the name of the course.

Weekly Plan (15 Weeks)



[Back to DBMS Page](#) | ● Not Connected Waiting for connection. Test Connection Close ^

Enter Connection String Below

psql 'postgres://USER:PASSWORD@HOST:PORT/DATABASE?sslmode=require'

Enter Your Student ID

e.g. 210123456

Browser Refresh

Tables 0

No tables loaded.

Relationships 0

No relationships loaded.

Editor Run i

Status	Rows	Time	Scripts
-	-	-	-

Comment Verify Clear

Write SQL here

Result Clear

No result.

SQL STUDIO



Back to DBMS Page

Not Connected

Waiting for connection.

Test Connection

Close

Back to DBMS Page

Enter Connection String Below

psql 'postgres://avnadmin:AVNS_...@pg-ie-326-ie-326.i.aivencloud.com:13140/defaultdb?sslmode=require'

Enter Your Student ID

.....

Browser

Refresh

Tables 0

No tables loaded.

Relationships 0

No relationships loaded.

Editor

Run

i

Status	Rows	Time	Scripts
-	-	-	-

Write SQL here

Comment

Verify

Clear

Result

Clear

No result.

SQL STUDIO

The screenshot shows the SQL Studio interface with a successful connection. At the top, there is a status bar with a green dot and the text "Connected." A red arrow points to this status indicator. To the left of the status bar is a button labeled "Back to DBMS Page". To the right are buttons for "Test Connection" and "Open".

The main interface is divided into three sections:

- Browser:** Contains two sections: "Tables" and "Relationships". Both sections show "0" items and a message "No tables loaded." and "No relationships loaded." respectively. A "Refresh" button is located at the top right of the Browser section.
- Editor:** Contains a large text area for writing SQL. Above the text area is a "Run" button. To the right of the "Run" button is a status bar showing "Status: Success", "Rows: 1", "Time: 14 ms", and "Scripts: 1". There are also buttons for "Comment", "Verify", and "Clear".
- Result:** Contains a table with the following data:

current_database name	current_user name	now timestampz
defaultdb	avnadmin	2026-03-22 19:59:28.150364+00

Below the table is a "Rows per page" dropdown set to "20". At the bottom right are buttons for "Previous", "Page 1 / 1", and "Next".

Any issue with
the connection?
Send me an
email

SQL STUDIO-CONNECTION SUCCESS



TABLE BROWSER

Back to DBMS Page

● Not Connected

Waiting for connection.

Test Connection

Close

Enter Connection String Below

psql 'postgres://USER:PASSWORD@HOST:PORT/DATABASE?sslmode=require'

Enter Your Student ID

e.g. 210123456

Browser

Refresh

Tables 0

No tables loaded.

Relationships 0

No relationships loaded.

Editor

Run

i

Status	Rows	Time	Scripts
-	-	-	-

Write SQL here

Comment

Verify

Clear

Result

Clear

No result.

RUN BUTTON

QUERY RESULTS PANE

SQL STUDIO

Introduction-Recap

With the PostgreSQL you can:

- Execute a query.
- Retrieve data.
- Add records.
- Update records.
- Delete records.
- Create new tables.
- Create stored procedures.
- Create Functions.
- Create new databases.
-

Introduction-Recap

- Some important command for SQL:

SELECT	Retrieves data from DB	
UPDATE	Updates data in a table.	
DELETE	Deletes data from a table.	
INSERT INTO	Adds data to a table.	
CREATE	TABLE, DATABASE, INDEX, TRIGGER, STORED PROCEDURE, FUNCTION	Creates new.
ALTER		Modifies the existing one.
DROP		Deletes the existing one.

Introduction-Recap

SQL commands are divided into three basic classes according to the operations they perform:

❑ Data manipulation language (DML):

- SELECT, INSERT, UPDATE etc. commands that operate on the data in the table.

❑ Data definition language (DDL):

- Including the commands used to define the data element starting with CREATE, ALTER, DROP.

❑ Data control language (DCL):

- GRANT, REVOKE etc. These commands for creating user privileges.

DATA DEFINITION LANGUAGE



PostgreSQL Data Types

PostgreSQL supports the following data types:

- Boolean
- Character types such as char, varchar, and text.
- Numeric types such as integer and floating-point number.
- Temporal types such as date, time, timestamp, and interval
- UUID for storing Universally Unique Identifiers
- Array for storing array strings, numbers, etc.
- JSON stores JSON data
- hstore stores key-value pair
- Special types such as network address and geometric data.

PostgreSQL Data Types-Boolean

A Boolean data type can hold one of three possible values: true, false, or null. You use `boolean` or `bool` keyword to declare a column with the Boolean data type.

When you insert data into a Boolean column, PostgreSQL converts it to a Boolean value

- `1`, `yes`, `y`, `t`, `true` values are converted to `true`
- `0`, `no`, `false`, `f` values are converted to `false`.

When you select data from a Boolean column, PostgreSQL converts the values back e.g., `t` to true, `f` to `false` and `space` to `null`.

PostgreSQL Data Types-Character

PostgreSQL provides three character data types: `CHAR(n)`, `VARCHAR(n)`, and `TEXT`

- `CHAR(n)` is the fixed-length character with space padded. If you insert a string that is shorter than the length of the column, PostgreSQL pads spaces. If you insert a string that is longer than the length of the column, PostgreSQL will issue an error.
- `VARCHAR(n)` is the variable-length character string. The `VARCHAR(n)` allows you to store up to `n` characters. PostgreSQL does not pad spaces when the stored string is shorter than the length of the column.
- `TEXT` is the variable-length character string. Theoretically, text data is a character string with unlimited length.

PostgreSQL Data Types-Numeric

Integer

There are three kinds of integers in PostgreSQL:

- Small integer (`SMALLINT`) is a 2-byte signed integer that has a range from -32,768 to 32,767.
- Integer (`INT`) is a 4-byte integer that has a range from -2,147,483,648 to 2,147,483,647.
- Serial is the same as integer except that PostgreSQL will automatically generate and populate values into the `SERIAL` column. This is similar to `AUTO_INCREMENT` ↗ column in MySQL or `AUTOINCREMENT` ↗ column in SQLite.

PostgreSQL Data Types-Numeric

Floating-point number

There are three main types of floating-point numbers:

- `float(n)` is a floating-point number whose precision, is at least, n, up to a maximum of 8 bytes.
- `real` or `float8` is a 4-byte floating-point number.
- `numeric` or `numeric(p,s)` is a real number with p digits with s number after the decimal point. This `numeric(p,s)` is the exact number.

PostgreSQL Data Types-Temporal

The temporal data types allow you to store date and /or time data. PostgreSQL has five main temporal data types:

- `DATE` stores the dates only.
- `TIME` stores the time of day values.
- `TIMESTAMP` stores both date and time values.
- `TIMESTAMPTZ` is a timezone-aware timestamp data type. It is the abbreviation for timestamp with the time zone.
- `INTERVAL` stores periods.

The `TIMESTAMPTZ` is PostgreSQL's extension to the SQL standard's temporal data types.

PostgreSQL Data Types-Others

Arrays

In PostgreSQL, you can store an array of strings, an array of integers, etc., in array columns. The array comes in handy in some situations e.g., storing days of the week, and months of the year.

JSON

PostgreSQL provides two JSON data types: `JSON` and `JSONB` for storing JSON data.

The `JSON` data type stores plain JSON data that requires reparsing for each processing, while `JSONB` data type stores `JSON` data in a binary format which is faster to process but slower to insert. In addition, `JSONB` supports indexing, which can be an advantage.

UUID

The `UUID` data type allows you to store Universal Unique Identifiers defined by RFC 4122 [↗](#). The `UUID` values guarantee a better uniqueness than `SERIAL` and can be used to hide sensitive data exposed to the public such as values of `id` in URL.

CREATE COMMAND



Introduction to PostgreSQL CREATE TABLE statement

Typically, a relational database consists of multiple related tables. Tables allow you to store structured data like customers, products, and employees.

To create a new table, you use the `CREATE TABLE` statement. Here's the basic syntax of the `CREATE TABLE` statement:

```
CREATE TABLE [IF NOT EXISTS] table_name (  
    column1 datatype(length) column_constraint,  
    column2 datatype(length) column_constraint,  
    ...  
    table_constraints  
);
```



CREATE COMMAND

First, specify the name of the table that you want to create after the `CREATE TABLE` keywords. The table name must be unique in a schema. If you create a table with a name that already exists, you'll get an error.

A schema is a named collection of database objects including tables. If you create a table without a schema, it defaults to public. You'll learn more about the schema in the schema tutorial.

Second, use the `IF NOT EXISTS` option to create a new table only if it does not exist. When you use the `IF NOT EXISTS` option and the table already exists, PostgreSQL will issue a notice instead of an error.

Third, specify table columns separated by commas. Each column definition consists of the column name, data type, size, and constraint.

CREATE COMMAND

The constraint of a column specifies a rule that is applied to data within a column to ensure data integrity. The column constraints include primary key, foreign key, not null, unique, check, and default.

For example, the `NOT NULL` constraint ensures that the values in a column cannot be NULL.

Finally, specify constraints for the table including primary key, foreign key, and check constraints.

A table constraint is a rule that is applied to the data within the table to maintain data integrity.

Note that some column constraints can be defined as table constraints such as primary key, foreign key, unique, and check constraints.

CREATE COMMAND-CONSTRAINTS

PostgreSQL includes the following column constraints:

- NOT NULL – ensures that the values in a column cannot be `NULL`.
- UNIQUE – ensures the values in a column are unique across the rows within the same table.
- PRIMARY KEY – a primary key column uniquely identifies rows in a table. A table can have one and only one primary key. The primary key constraint allows you to define the primary key of a table.
- CHECK – ensures the data must satisfy a boolean expression. For example, the value in the price column must be zero or positive.
- FOREIGN KEY – ensures that the values in a column or a group of columns from a table exist in a column or group of columns in another table. Unlike the primary key, a table can have many foreign keys.

Table constraints are similar to column constraints except that you can include more than one column in the table constraint.

CREATE COMMAND- EXAMPLE COMMAND

We will create a new table called `accounts` in the sample database. The `accounts` table has the following columns:

- `user_id` – primary key
- `username` – unique and not null
- `password` – not null
- `email` – unique and not null
- `created_at` – not null
- `last_login` – null

CREATE COMMAND- EXAMPLE COMMAND

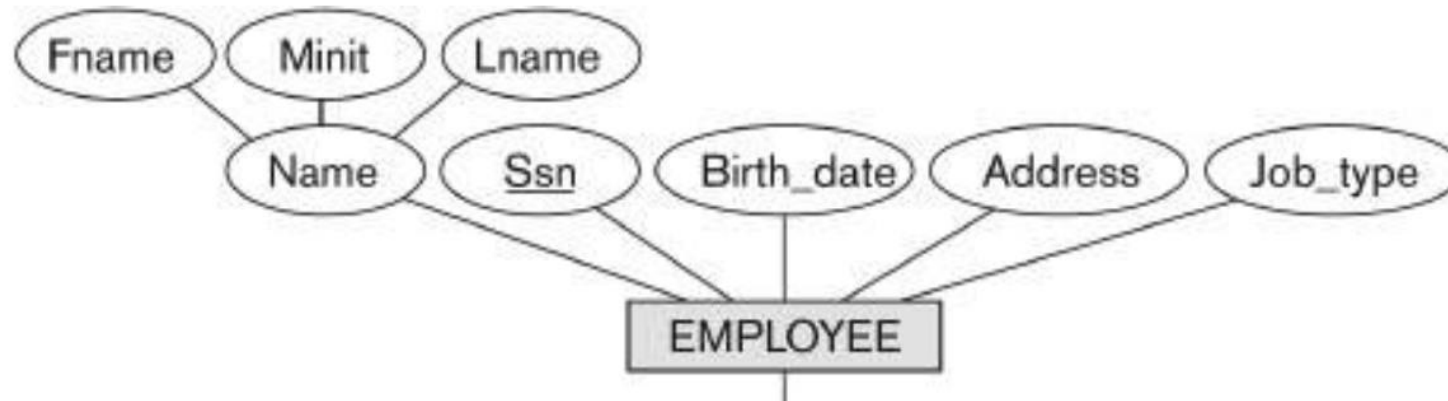
The following example uses the `CREATE TABLE` statement to create the `accounts` table:

```
CREATE TABLE accounts (  
  user_id SERIAL PRIMARY KEY,  
  username VARCHAR (50) UNIQUE NOT NULL,  
  password VARCHAR (50) NOT NULL,  
  email VARCHAR (255) UNIQUE NOT NULL,  
  created_at TIMESTAMP NOT NULL,  
  last_login TIMESTAMP  
);
```



CREATE COMMAND- WORK ON YOUR OWN

Now create a table for the previous week ER diagram



CREATE COMMAND- SOLUTION

```
-- Create the employee table based on the ER schema
CREATE TABLE employees (
    -- Ssn is underlined in the ER diagram, indicating it is the Primary Key.
    -- Using CHAR(9) or VARCHAR(11) is common for SSN;
    -- we'll use VARCHAR for flexibility.
    ssn VARCHAR(11) PRIMARY KEY,

    -- The 'Name' composite attribute is flattened into its sub-parts:
    fname VARCHAR(50) NOT NULL,
    minit CHAR(1),
    lname VARCHAR(50) NOT NULL,

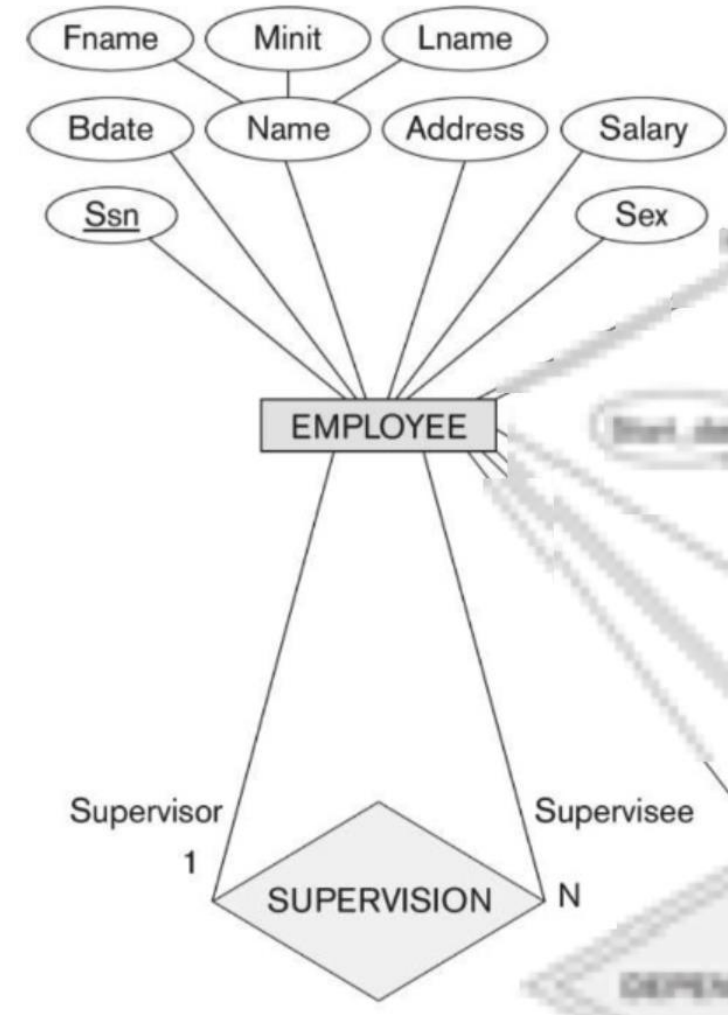
    -- Standard date type for birth dates
    birth_date DATE,

    -- Address can be long, so VARCHAR(255) or TEXT is appropriate
    address VARCHAR(255),

    -- Job_type represents a category or title
    job_type VARCHAR(50)
);
```

CREATE COMMAND- WORK ON YOUR OWN

Now create a table for the previous week ER diagram



CREATE COMMAND- EXAMPLE COMMAND

```
CREATE TABLE employee (  
    fname VARCHAR(50) NOT NULL,  
    minit CHAR(1),  
    lname VARCHAR(50) NOT NULL,  
    ssn CHAR(9) PRIMARY KEY,  
    bdate DATE,  
    address VARCHAR(100),  
    sex CHAR(1),  
    salary DECIMAL(10, 2),  
    super_ssn CHAR(9) REFERENCES employee(ssn), -- Recursive Supervision  
    dno INT REFERENCES department(dnumber)      -- Works_for relationship  
);
```

CREATE COMMAND- Insert DATA

-- Inserting data into the flattened schema

```
INSERT INTO employees (ssn, fname, minit, lname, birth_date, address, job_type)
VALUES
('987-65-4321', 'John', 'B', 'Smith', '1985-01-15', '123 Maple St, Ankara', 'Engineer'),
('123-45-6789', 'Alice', 'M', 'Johnson', '1992-05-22', '456 Oak Ave, Istanbul', 'Manager'),
('555-00-1212', 'Robert', NULL, 'Taylor', '1978-11-30', '789 Pine Rd, Izmir', 'Technician');
```

-- Confirming the data

```
SELECT * FROM employees;
```

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM